If Statements In Java

Generated Aug 12, 2026 7:59 AM

What We'll Learn

By the end of this lesson, students can:

- Explain how an `if` statement controls program flow.
- Write an `if` statement using a Boolean condition.
- Identify when the statements inside an `if` block will run.
- Trace the output of programs that use `if` statements.

Before We Start

Consider this classroom rule:

> If a student submits the activity, record the activity as completed.

What happens when the student submits the activity? What happens when the student does not submit it?

Java programs can use an `if` statement to perform an action only when a condition is `true`.

Let's Understand

An `if` statement allows a Java program to make a decision.

It checks a Boolean condition and runs a block of code only when the condition produces `true`.

Basic Syntax

```
if (condition) {
    // Statements that run when the condition is true
}
```

The condition is written inside parentheses.

The statements controlled by the condition are written inside curly braces.

Example:

```
int score = 85;

if (score >= 75) {
    System.out.println("You passed.");
}
```

The condition is:

```
score >= 75
```

Since `85` is greater than or equal to `75`, the condition is `true`.

Output:

```
You passed.
```

When the Condition Is False

Consider the following example:

```
int score = 70;

if (score >= 75) {
    System.out.println("You passed.");
}
```

The condition `score >= 75` is `false`.

The program skips the statement inside the `if` block, so nothing is displayed.

Parts of an `if` Statement

```
if (temperature >= 30) {
    System.out.println("The weather is hot.");
}
```

The statement contains the following parts:

- `if` is the Java keyword used to begin the decision.
- `temperature >= 30` is the Boolean condition.
- { and } identify the block of code controlled by the condition.
- `System.out.println()` runs only when the condition is `true`.

Using Different Comparison Operators

An `if` statement can use different comparison operators.

```
if (age > 18) {
    System.out.println("Age is greater than 18.");
}
```

```
if (price < 100) {
    System.out.println("The item costs less than 100.");
}
```

```
if (quantity == 5) {
    System.out.println("The quantity is exactly 5.");
}
```

```
if (score != 0) {
    System.out.println("The score is not zero.");
}
```

Remember:

- Use = to assign a value.
- Use == to compare two values.

Program Flow

Java normally runs statements from top to bottom.

When Java reaches an `if` statement:

1. It checks the condition.
2. If the condition is `true`, it runs the statements inside the block.
3. If the condition is `false`, it skips the block.
4. It continues with the next statement after the `if` block.

Example:

```
int number = 10;

System.out.println("Checking the number...");

if (number > 0) {
    System.out.println("The number is positive.");
}

System.out.println("Checking is complete.");
```

Output:

```
Checking the number...
The number is positive.
Checking is complete.
```

Let's Practice

Create a Java program named `DiscountChecker`.

The program must:

1. Store the purchase amount in a `double` variable.
2. Check whether the purchase amount is greater than or equal to `1000`.
3. Display `You qualify for a discount.` when the condition is `true`.
4. Display the purchase amount after the `if` statement.
5. Test the program using two different purchase amounts.